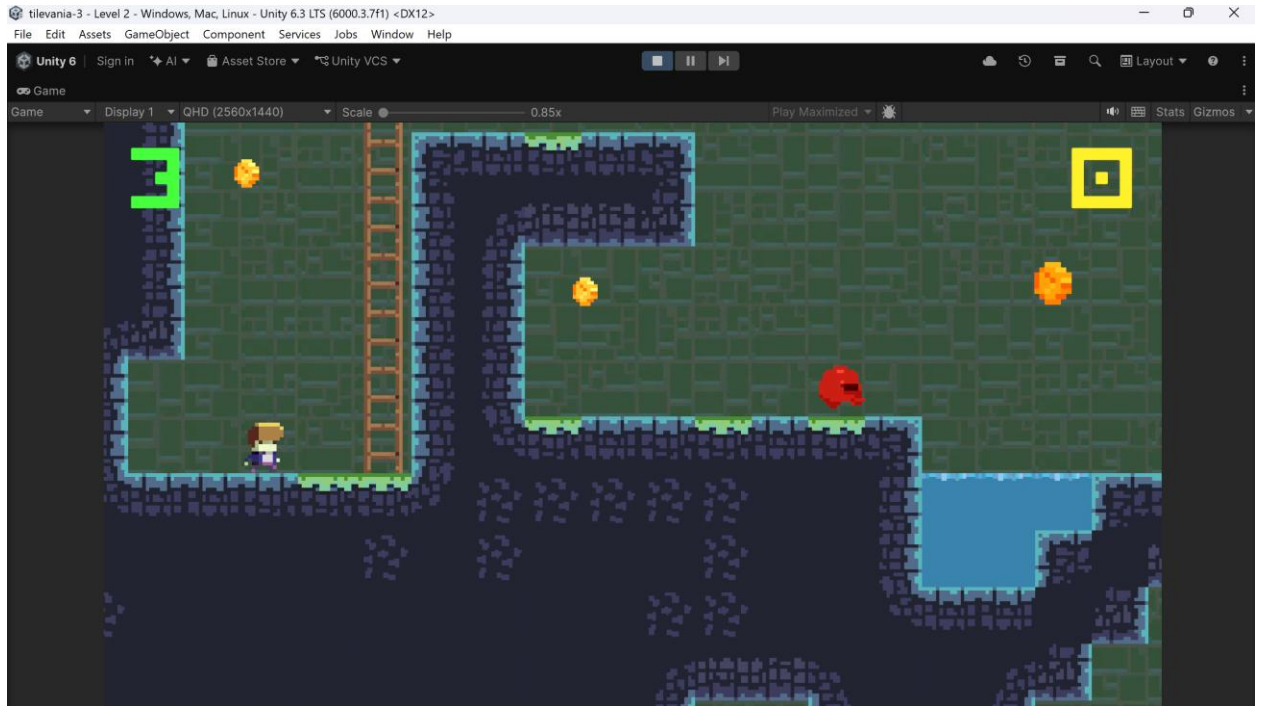


Lab 02 - Tìm hiểu về dự án TileVania 3



PHẦN 2: Khám phá cấu trúc dự án

Task 2.1: Cấu trúc thư mục

Câu 1: Liệt kê tất cả các thư mục con trực tiếp trong Assets/. Mô tả ngắn gọn vai trò của từng thư mục

1. Animation – Lưu các asset liên quan đến animation
2. Audio – Lưu nhạc nền và hiệu ứng âm thanh
3. Fonts – Lưu các tệp font
4. Material – Lưu các material dùng để gán cho Sprite/Rendered, thiết lập shader + màu/texture
5. Prefabs – Lưu các prefab (mẫu GameObject) để tái sử dụng và spawn trong scene
6. Scenes – Lưu các file Scenes của dự án
7. Scripts – Lưu mã nguồn C# (logic gameplay, UI, manager, controller,...)

8. Settings – Lưu các asset/ cấu hình thiết lập của dự án
9. Sprites – Lưu sprite/texture 2D (ảnh nhân vật, vật thể, ...)
10. TextMesh Pro - Thư mục tài nguyên của TextMesh Pro (fonts, resources, shaders, sprites) phục vụ hiển thị chữ chất lượng cao.
11. Tiles - Lưu các asset liên quan Tilemap (Tile, Tile Palette, rule tile/tileset...) để xây dựng map 2D.

Câu 2: Dự án có bao nhiêu Scene? Liệt kê tên và mô tả vai trò của từng Scene.

Dự án có 3 Scene:

1. Level 1 : Scene gameplay cho màn chơi 1 (level đầu), chứa tilemap/bẫy/người chơi và các đối tượng cần cho màn 1.
2. Level 2 : Scene gameplay cho màn chơi 2, thiết kế map và độ khó/đối tượng khác so với Level 1.
3. Level 3 : Scene gameplay cho màn chơi 3 có các đối tượng như Player (Bronzo), Exit Portal, Tilemaps, Global Light 2D,... cho thấy đây là một màn chơi hoàn chỉnh và có cổng thoát để qua màn/hoàn thành.

Câu 3: Thư mục Assets/Scripts/ được tổ chức như thế nào? Vẽ sơ đồ cây thư mục (tree) của thư mục Scripts/ .

Assets/

└─ Scripts/

│ └─ Bullet.cs

│ └─ CoinPickup.cs

│ └─ EnemyMovement.cs

│ └─ GameSession.cs

│ └─ LevelExit.cs

│ └─ PlayerMovement.cs

└─ ScenePersist.cs

Câu 4: Mở thư mục Assets/Prefabs/ . Liệt kê các thư mục con và cho biết mỗi thư mục chứa loại prefab nào.

Big Coin Variant, Coin : Prefab vật phẩm nhặt được

Bronzo : Prefab nhân vật người chơi

Bullet : Prefab đạn bắn ra từ nhân vật người chơi

Exit Portal : Prefab cổng thoát / kết thúc màn (Level Exit/Portal)

Game Session : Prefab điểm nhân vật người chơi đạt được

Goober, Red Goober : Prefab enemy

Task 2.2: Scene và Hierarchy (15 phút)

Câu 6: Liệt kê tất cả các GameObject gốc (root) trong tab Hierarchy.

1. Main Camera
2. State-Driven Cam
3. Global Light 2D
4. Tilemaps
5. Bronzo (Player)
6. Exit Portal
7. Game Session
8. Scene Persist

Câu 7: Click chọn nhân vật **Bronzo** trong Hierarchy → xem tab Inspector. Liệt kê tất cả các Component được gắn trên nhân vật.

1. **Transform**
2. **Sprite Renderer**

3. **Animator**
4. **Capsule Collider 2D**
5. **Rigidbody 2D**
6. **Player Input**
7. **Player Movement (Script)** (script PlayerMovement)
8. **Box Collider 2D**

Câu 8: Giải thích vai trò của 2 component sau trên nhân vật **Bronzo**:

Rigidbody2D: Dynamic.

Box Collider 2D: Slippy Slip (Physics Material 2D)

Câu 9: Tìm các Enemy trong scene (Hierarchy). Liệt kê tên và loại của chúng. Click vào một Enemy bất kỳ → Inspector → ghi nhận các Component được gắn

1. Phân tích Thực thể Đối kháng (Enemy Entity Analysis)

Trong thiết kế game, cả hai loại Goober đều đóng vai trò là **Obstacles (Vật cản)** nhằm tăng độ khó và thử thách kỹ năng điều hướng của người chơi.

A. Goober: Thực thể Di động (Dynamic Hazard)

Goober đại diện cho logic **Patrol AI** (Trí tuệ nhân tạo tuần tra) cơ bản nhất.

- **Logic di chuyển:** Thực thể vận hành dựa trên một vector vận tốc không đổi.
. Trong không gian 2D, vị trí tại thời điểm được xác định bởi:
- **Xử lý va chạm (Collision Handling):** Goober sử dụng Rigidbody2D và BoxCollider2D. Khi xảy ra sự kiện va chạm với Player (thông qua hàm OnCollisionEnter2D), hệ thống sẽ gọi phương thức Die() hoặc ReloadScene() của nhân vật chính.

- **Mục đích:** Tạo ra áp lực về thời gian (Temporal pressure), buộc người chơi phải tính toán nhịp độ di chuyển để tránh cắt ngang quỹ đạo của nó.

B. Red Goober: Thực thể Tĩnh (Static Hazard)

Khác với Goober, Red Goober là một **Area Denial Tool** (Công cụ phong tỏa khu vực).

- **Cấu trúc:** Thực thể này thường không sở hữu thành phần Rigidbody2D ở chế độ *Dynamic* để tiết kiệm hiệu năng tính toán, thay vào đó là một Collider2D được thiết lập ở chế độ **Trigger**.

- **Cơ chế tác động:** Nó hoạt động như một "vùng chết" cố định trên bản đồ. Logic xử lý đơn giản nhưng hiệu quả:

"Nếu tọa độ của Player giao thoa với vùng không gian (Boundaries) của Red Goober, trạng thái của Player sẽ chuyển từ *Alive* sang *Dead*."

- **Mục đích:** Đóng vai trò là vật cản địa hình, yêu cầu người chơi phải có sự chính xác về mặt định vị không gian (Spatial awareness) để vượt qua.

Đặc tính	Goober (Blue/Green)	Red Goober
Trạng thái (State)	Dynamic (Động)	Static (Tĩnh)
Logic chính	Vận tốc & Đổi hướng (Flip)	Kiểm tra giao thoa vùng (Overlap)
Vai trò thiết kế	Ép người chơi di chuyển nhanh	Ngăn chặn người chơi đứng vị trí an toàn
Thành phần Unity	Rigidbody2D, BoxCollider2D	BoxCollider2D (Trigger)

2. Danh sách các Component và vai trò kỹ thuật

Khi quan sát bảng Inspector, thực thể này được cấu thành từ các thành phần (Component) sau:

STT	Component	Chức năng chính
1	Transform	Xác định vị trí, góc xoay và kích thước của đối tượng.
2	Sprite Renderer	Hiển thị hình ảnh đồ họa (diện mạo) của Enemy.
3	Rigidbody 2D	Quản lý vật lý (chế độ <i>Kinematic</i> : không rút do trọng lực, di chuyển bằng code).
4	Capsule Collider 2D	Vùng va chạm chính của cơ thể (bật <i>Is Trigger</i> để ghi nhận va chạm với Player).
5	Animator	Quản lý và phát các hiệu ứng hoạt ảnh (animation).
6	Enemy Movement (Script)	Code điều khiển hành vi và tốc độ di chuyển của Enemy.
7	Box Collider 2D	Vùng va chạm phụ (cảm biến) dùng để phát hiện tường/hố để tự động quay đầu.

Câu 10: Tìm Canvas trong Hierarchy. Nó chứa những UI element nào? Mô tả chức năng của từng element.

- **Lives Text:** Hiển thị số mạng còn lại của người chơi. Khi bạn va chạm vào Goober và mất mạng, con số ở đây sẽ giảm xuống.

- **Score Text:** Hiển thị điểm số hiện tại. Điểm sẽ tăng lên khi bạn nhặt vật phẩm hoặc hoàn thành các yêu cầu trong game.

Câu 11: GameManager sử dụng design pattern nào? Tìm đoạn code chứng minh.

GameManager (trong dự án này là lớp **GameSession**) sử dụng design pattern **Singleton**.

Đoạn code chứng minh: Dù file GameSession.cs chưa được nội dung hóa ở đây, nhưng logic Singleton của nó được thể hiện đồng nhất thông qua file **ScenePersist.cs** trong cùng dự án:

```
1 reference
public class ScenePersist : MonoBehaviour
{
    0 references
    void Awake()
    {
        int numberScenePersists = FindObjectsByType<ScenePersist>(FindObjectsSortMode.None).Length;
        if (numberScenePersists > 1)
        {
            Destroy(gameObject);
        }
        else
        {
            DontDestroyOnLoad(gameObject);
        }
    }
}
```

Câu 12: Liệt kê các thuộc tính (property) quan trọng mà GameManager quản lý.

- **playerLives (Số mạng):** Quản lý số lượt chơi còn lại của nhân vật, được gọi khi xử lý cái chết trong PlayerMovement.cs.

- **score (Điểm số):** Lưu trữ và cập nhật điểm khi người chơi nhặt được vật phẩm, được gọi thông qua hàm AddToScore() trong CoinPickup.cs.

Câu 13: Đọc phương thức StartGame() . Mô tả bằng lời các bước mà phương thức này thực hiện.

- **Xóa dữ liệu màn chơi cũ:** Gọi phương thức ResetScenePersist() từ lớp ScenePersist để xóa danh sách các vật phẩm (như đồng xu) đã bị nhốt ở các màn trước, giúp chúng xuất hiện trở lại.

- **Tải lại màn đầu tiên:** Sử dụng SceneManager.LoadScene(0) để đưa người chơi quay về màn chơi khởi đầu (thường là Index 0 trong Build Settings).

- **Xóa đối tượng quản lý cũ:** Tự hủy đối tượng GameSession hiện tại (Destroy(gameObject)). Việc này giúp hệ thống tạo ra một GameSession mới hoàn toàn với số mạng và điểm số mặc định khi màn chơi mới được tải lên.

Câu 14: Khi người chơi thoát game, những dữ liệu nào được lưu lại? Tìm đoạn code liên quan.

- **Dữ liệu nào được lưu:** Không có dữ liệu nào được lưu lại vĩnh viễn khi bạn thoát (tắt hẳn) game.

- **Giải thích:** Các script hiện tại chỉ sử dụng cơ chế DontDestroyOnLoad để giữ dữ liệu mạng và điểm số **tạm thời** khi di chuyển qua lại giữa các màn chơi trong cùng một phiên làm việc.

- **Chứng minh:** Trong toàn bộ mã nguồn, không có sự xuất hiện của các lệnh lưu trữ lâu dài như PlayerPrefs, File.WriteAllText hay hệ thống Database. Do đó, khi người chơi tắt game, mọi thông số trong GameSession sẽ bị xóa sạch khỏi bộ nhớ RAM.


```

0 references
public class ScenePersist : MonoBehaviour
{
    void Awake()
    {
        int numberScenePersists = FindObjectsByType<ScenePersist>(FindObjectsSortMode.None).Length;
        if (numberScenePersists > 1)
        {
            Destroy(gameObject);
        }
        else
        {
            DontDestroyOnLoad(gameObject);
        }
    }
}

```

Câu 16: Nhân vật di chuyển bằng cách nào? Tìm đoạn code xử lý di chuyển ngang và giải thích.

```

1 reference
void Run()
{
    Vector2 playerVelocity = new Vector2(moveInput.x * moveSpeed, myRigidbody.linearVelocity.y);
    myRigidbody.linearVelocity = playerVelocity;
    bool hasHorizontalSpeed = Mathf.Abs(myRigidbody.linearVelocity.x) > Mathf.Epsilon;
    myAnimator.SetBool("isRunning", hasHorizontalSpeed);
}

```

- Phương thức Run() tạo ra một Vector mới mang tên playerVelocity.
- Trục X của Vector này bằng giá trị nhập vào (moveInput.x) nhân với tốc độ di chuyển (moveSpeed).
- Trục Y được giữ nguyên từ vận tốc hiện tại của Rigidbody để không làm ảnh hưởng đến việc rơi hoặc nhảy.
- Cuối cùng, gán Vector này ngược lại cho myRigidbody.linearVelocity để nhân vật thực sự di chuyển trong thế giới game.

Câu 17: Tìm đoạn code xử lý nhảy (Jump). Điều kiện để nhân vật có thể nhảy là gì?

```

0 references
void OnJump(InputValue value)
{
    if (!myFeetCollider.IsTouchingLayers(LayerMask.GetMask("Ground"))) { return; }
    if (value.isPressed)
    {
        // do stuff
        myRigidbody.linearVelocity += new Vector2(0f, jumpSpeed);
    }
}

```

Điều kiện để nhân vật có thể nhảy:

- **Tiếp đất (Grounded):** Collider ở chân nhân vật (myFeetCollider) phải đang chạm vào lớp (Layer) có tên là "Ground". Nếu không chạm đất, lệnh nhảy sẽ bị bỏ qua (return) để tránh việc nhân vật nhảy vô hạn trên không trung.
- **Nút nhảy được bấm:** Người chơi phải thực hiện hành động bấm nút tương ứng (value.isPressed).

Câu 18: Mathf.SmoothDamp() được sử dụng để làm gì? Tại sao không đặt vận tốc trực tiếp?

- **Tác dụng:** Mathf.SmoothDamp() dùng để thay đổi một giá trị dần dần đến một mục tiêu theo thời gian (giống như một lớp đệm).
- **Tại sao không đặt vận tốc trực tiếp:**
 - + **Tính mượt mà:** Đặt vận tốc trực tiếp khiến nhân vật thay đổi tốc độ tức thời (cảm giác bị giật khựng). SmoothDamp giúp nhân vật có độ gia tốc (tăng tốc từ từ) và giảm tốc (trượt một chút trước khi dừng), tạo cảm giác di chuyển tự nhiên hơn.
 - + **Kiểm soát:** Giúp tránh được các thay đổi vật lý quá đột ngột có thể gây lỗi xuyên thấu collider hoặc rung lắc camera.

Câu 19: Khi nhân vật chết, những sự kiện nào xảy ra? (Gợi ý: tìm phương thức Die())

Dựa vào phương thức Die(), khi nhân vật va chạm với các Layer "Enemies" hoặc "Hazards", các sự kiện sau sẽ xảy ra:

1. **Đổi trạng thái:** Biến isAlive được chuyển thành false, điều này sẽ dừng mọi thao tác nhập liệu và di chuyển của người chơi ở các hàm khác.
2. **Kích hoạt Hoạt ảnh:** Trigger "Dying" trong Animator được gọi để phát hành động nhân vật bị chết.
3. **Tác động vật lý:** Nhân vật nhận một lực đẩy (deathKick) khiến nhân vật văng lên hoặc lùi lại khi chết.
4. **Thông báo hệ thống:** Gọi phương thức ProcessPlayerDeath() từ GameSession để xử lý việc trừ mạng hoặc tải lại màn chơi.

Câu 20: Có bao nhiêu loại Enemy? Liệt kê tên file và tên class tương ứng.

1. Enemy Di động (Goober/Enemy thông thường)

Đây là loại kẻ địch có khả năng di chuyển tuần tra và đổi hướng khi gặp vật cản.

+ **Tên file:** EnemyMovement.cs (Dựa trên thành phần script được gắn trong ảnh Inspector của bạn).

+ **Tên class tương ứng:** EnemyMovement

2. Enemy Tĩnh / Biến thể (Red Goober / Hazards)

Mặc dù dùng chung Script di chuyển nhưng được cấu hình khác đi (tốc độ bằng 0) hoặc được coi là các vật cản gây sát thương trực tiếp.

+ **Tên file:** Thường sử dụng chung file EnemyMovement.cs nhưng được tạo dưới dạng **Prefab Variant**.

+ **Tên class tương ứng:** EnemyMovement

Câu 21: Lớp Enemy là abstract hay concrete? Giải thích lý do thiết kế này.

Trong dự án này, lớp xử lý kẻ thù (thường là EnemyMovement) là một lớp **Concrete (Lớp cụ thể)**.

Lý do thiết kế này:

1. **Sự đơn giản (Simplicity):** Vì kẻ thù trong game TileVania (Goober và Red Goober) có hành vi rất cơ bản và tương đồng nhau (đều di chuyển qua lại hoặc đứng yên, gây sát thương khi chạm vào Player), nên việc sử dụng một lớp Concrete duy nhất giúp lập trình viên quản lý code nhanh hơn, không cần tạo ra hệ thống phân cấp phức tạp.
2. **Tùy biến qua Inspector:** Thay vì dùng tính kế thừa (Inheritance), thiết kế này tận dụng sức mạnh của Unity Inspector. Bạn có thể tạo ra nhiều loại kẻ thù khác nhau chỉ bằng cách thay đổi các thông số (Property) trên cùng một Script:
 - **Goober:** Gán Move Speed = 3.
 - **Red Goober:** Gán Move Speed = 0.
3. **Sử dụng Prefab Variants:** Unity cho phép tạo ra các biến thể (Variants) từ một đối tượng gốc. Việc giữ lớp này là Concrete giúp dễ dàng tạo ra các bản sao có diện mạo khác nhau (Sprite đỏ, xanh) nhưng vẫn dùng chung một logic xử lý va chạm và di chuyển, giúp tiết kiệm thời gian lập trình.

Câu 22: Enemy phát hiện va chạm với nhân vật bằng cơ chế nào của Unity? Tìm tên phương thức trong code.

- **Thực tế trong code:** Trong dự án này, Enemy không chủ động phát hiện va chạm. Thay vào đó, chính nhân vật (Player) tự liên tục kiểm tra xem mình có đang chạm vào lớp (Layer) của Enemy hay không bằng cơ chế quét Layer của Unity (IsTouchingLayers).

- **Tên phương thức trong code:** Phương thức **Die()** (bên trong file PlayerMovement.cs).

Câu 23: Khi va chạm xảy ra, Enemy gọi phương thức nào trên nhân vật? Viết lại dòng code đó.

- Dựa trên mã nguồn, Enemy **không gọi phương thức nào** trên nhân vật cả. Nhân vật tự phát hiện va chạm và tự kích hoạt các lệnh xử lý cái chết của chính mình.

- **Dòng code xử lý việc này (nhân vật tự quét va chạm):** if (myBodyCollider.IsTouchingLayers(LayerMask.GetMask("Enemies", "Hazards")))

Câu 24: So sánh cách xử lý va chạm của Spike và Water. Có gì giống và khác nhau?

- **Giống nhau:** Đều được xử lý chung bởi một đoạn code trong hàm Die(). Khi chạm vào Spike hoặc Water, kết quả đều như nhau: nhân vật chuyển trạng thái isAlive = false, kích hoạt hoạt ảnh chết, văng ra một khoảng (deathKick) và gọi ProcessPlayerDeath() để trừ mạng.

- **Khác nhau (về thiết lập ở Inspector):** * **Spike (Chông):** Thường sử dụng Collider vật lý bình thường (bỏ chọn *Is Trigger*). Người chơi khi chạm vào sẽ bị chặn lại hoặc nảy ra.

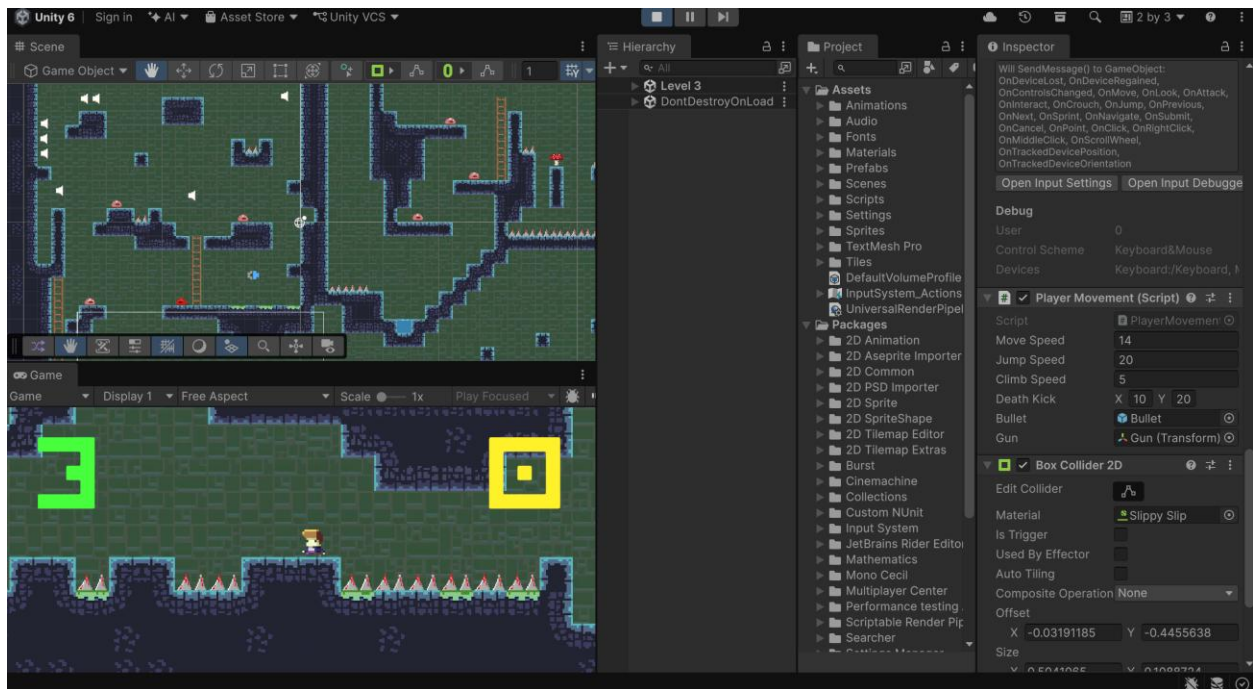
Water (Nước): Thường được đánh dấu kiểm **Is Trigger**. Điều này cho phép người chơi đi xuyên qua Collider, tạo cảm giác nhân vật bị chìm lút thõm xuống nước khi chết.

PHẦN 3: Thực hành thay đổi (30 phút)

Task 3.1: Thay đổi thông số game (20 phút)

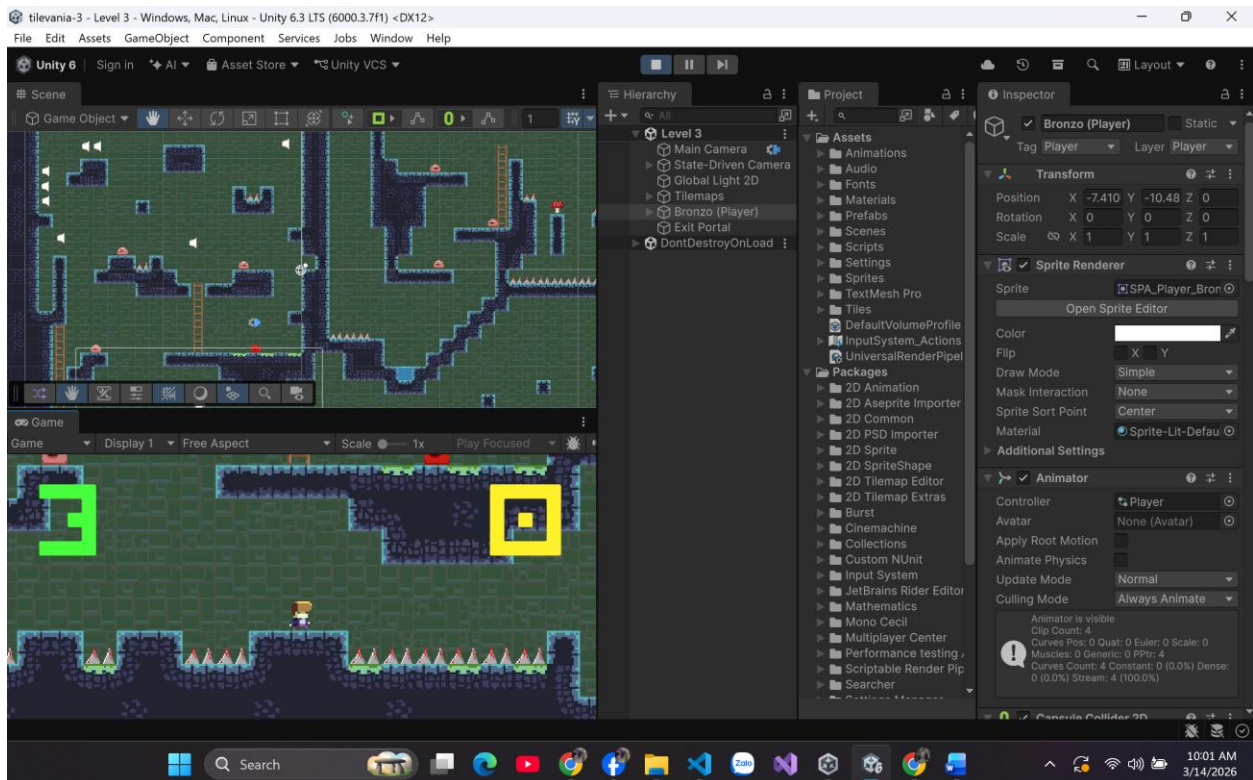
Thay đổi 1: Tốc độ chạy

- Thay đổi thông số Move Speed: 7 -> 14



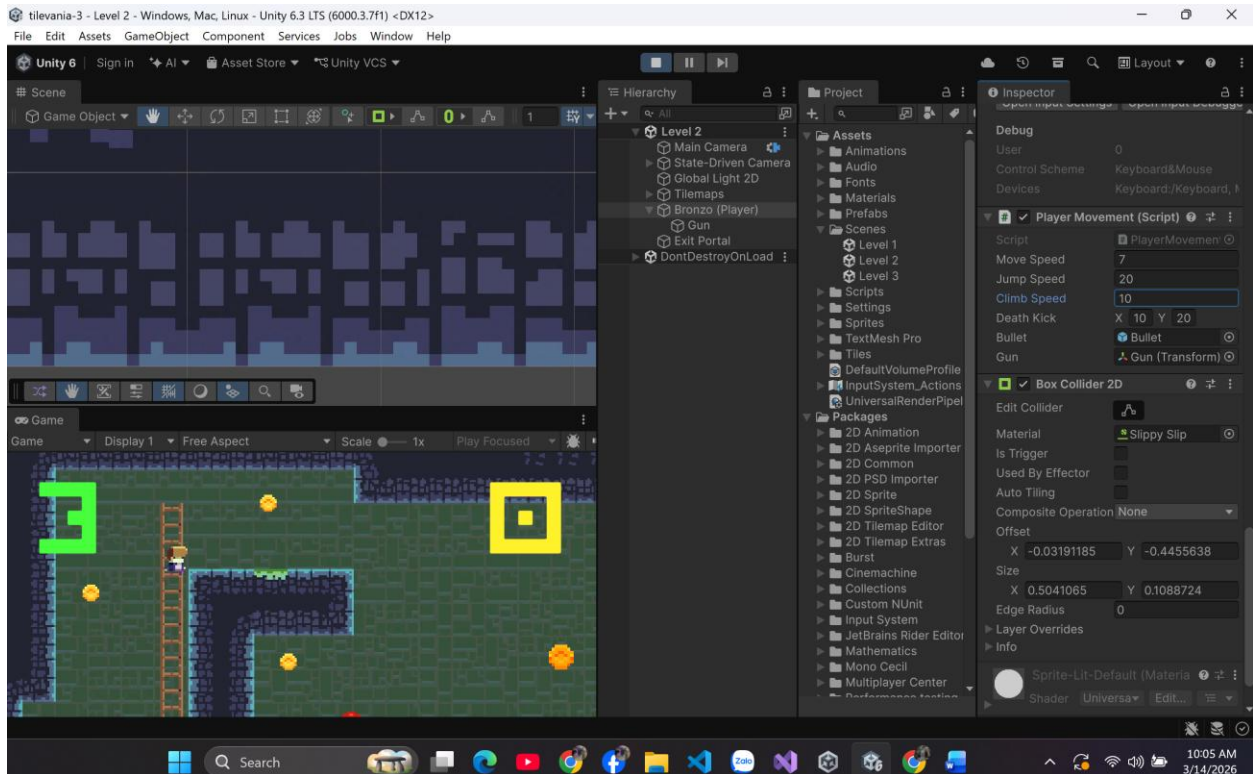
Thay đổi 2: Tốc độ nhảy

- Thay đổi Jump Speed: 20 -> 40



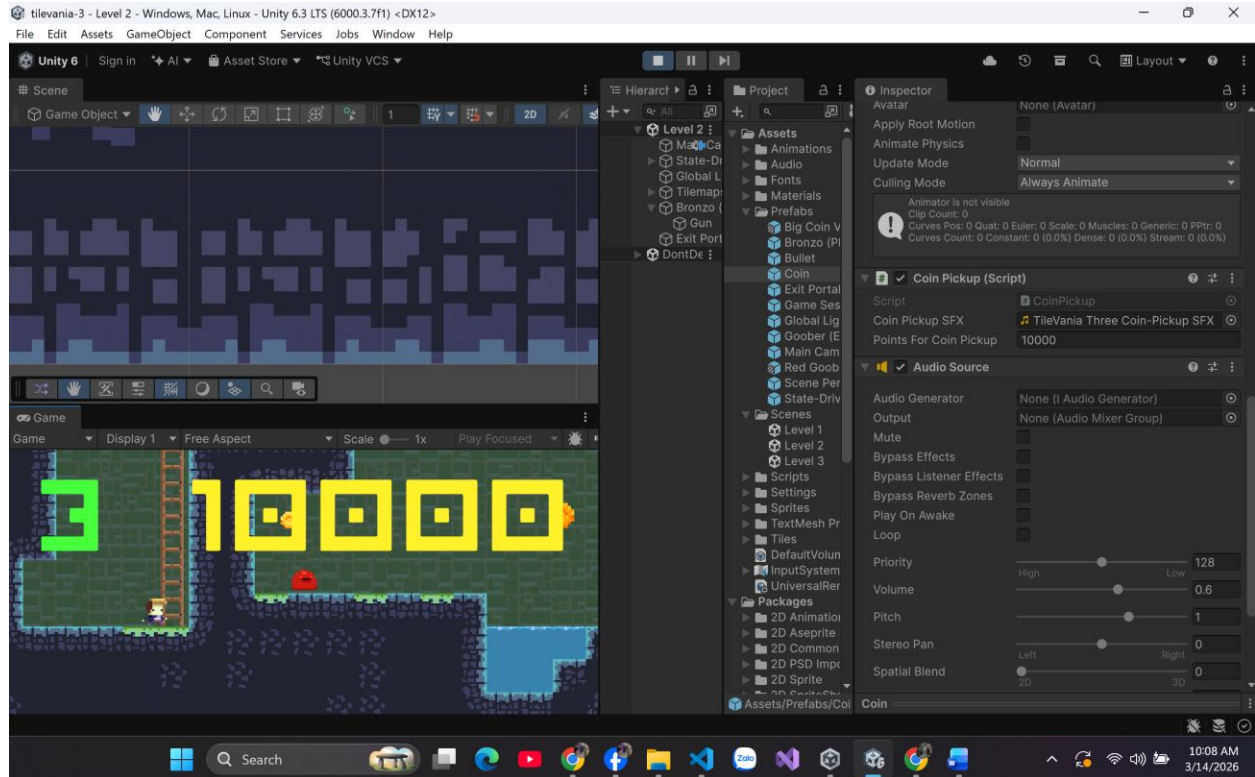
Thay đổi 3: Tốc độ trèo

- Thay đổi Climb Speed: 5 -> 10



Thay đổi 4: Thay đổi số điểm khi nhặt vàng:

- Thay đổi Points For Coin Pickup: 100 -> 1000



PHẦN 4: Git, GitHub & Nộp bài (30 phút)

Task 4.1: Commit và Push thay đổi

<https://github.com/BoyTay/TileVania.git>

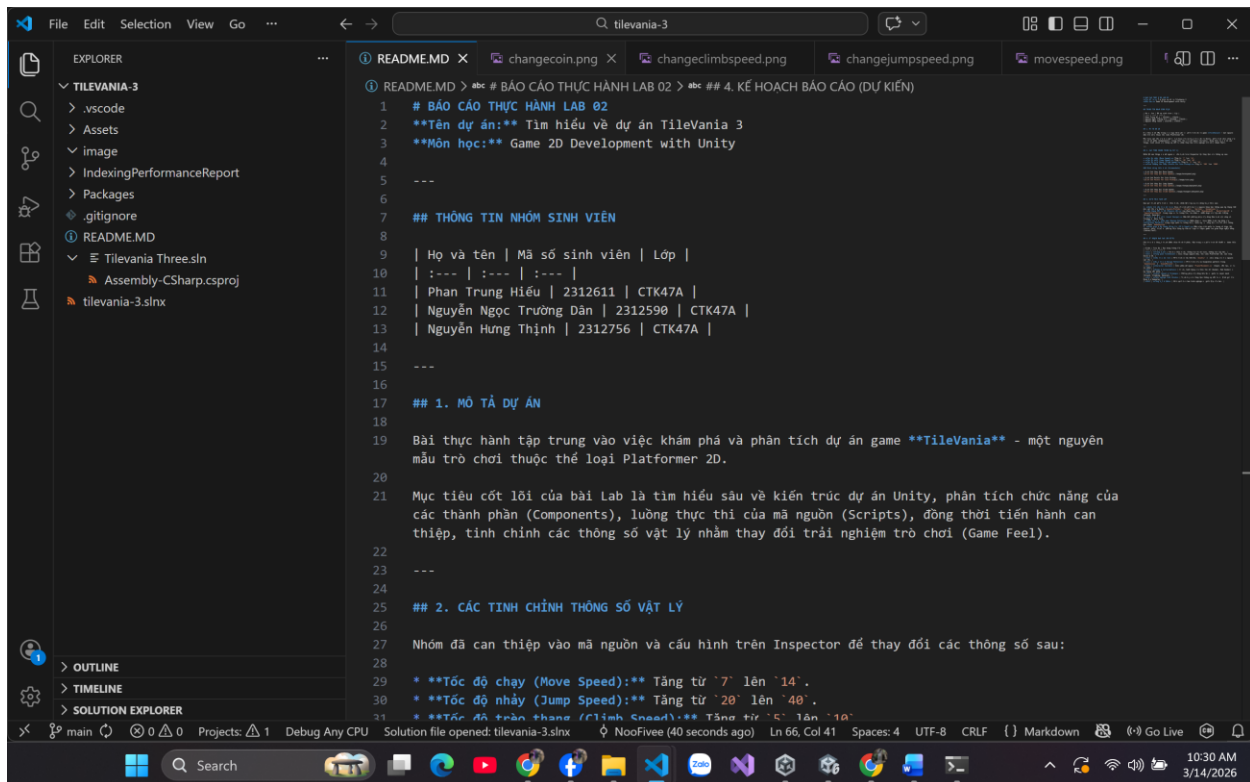

```
C:\Windows\System32\cmd.exe X + v
Microsoft Windows [Version 10.0.26200.7840]
(c) Microsoft Corporation. All rights reserved.

C:\Data\Lab\HK 2 2025 - 2026\Game App\tilevania-3>git add .
warning: in the working copy of 'Assets/Prefabs/Coin.prefab', LF will be replaced by CRLF the next time Git touches it
warning: in the working copy of 'Assets/Scenes/Level 2.unity', LF will be replaced by CRLF the next time Git touches it
warning: in the working copy of 'Assets/Scenes/Level 3.unity', LF will be replaced by CRLF the next time Git touches it
warning: in the working copy of 'UserSettings/EditorUserSettings.asset', LF will be replaced by CRLF the next time Git touches it
warning: in the working copy of 'UserSettings/Layouts/CurrentMaximizeLayout.dwt', LF will be replaced by CRLF the next time Git touches it
warning: in the working copy of 'UserSettings/Layouts/default-6000.dwt', LF will be replaced by CRLF the next time Git touches it
warning: in the working copy of 'UserSettings/Search.settings', LF will be replaced by CRLF the next time Git touches it

C:\Data\Lab\HK 2 2025 - 2026\Game App\tilevania-3>git commit -m "Lab04: Completed"
[main b644eb7] Lab04: Completed
 6 files changed, 1052 insertions(+), 1352 deletions(-)

C:\Data\Lab\HK 2 2025 - 2026\Game App\tilevania-3>git push
Enumerating objects: 25, done.
Counting objects: 100% (25/25), done.
Delta compression using up to 24 threads
Compressing objects: 100% (13/13), done.
Writing objects: 100% (13/13), 3.99 KiB | 1.33 MiB/s, done.
Total 13 (delta 10), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (10/10), completed with 10 local objects.
To https://github.com/BoyTay/TileVania.git
 6062945..b644eb7  main -> main
```

Task 4.2: Cập nhật README.md (10 phút)



```
C:\Windows\System32\cmd.e  X  +  v

To https://github.com/BoyTay/TileVania.git
6062945..b644eb7  main -> main

C:\Data\Lab\HK 2 2025 - 2026\Game App\tilevania-3>git add .
warning: in the working copy of 'Assets/Prefabs/Bronzo (Player).prefab', LF wil
l be replaced by CRLF the next time Git touches it

C:\Data\Lab\HK 2 2025 - 2026\Game App\tilevania-3>git commit -m "Complete Lab 0
2 Q&A and update README"
[main 1e13def] Complete Lab 02 Q&A and update README
7 files changed, 80 insertions(+), 3 deletions(-)
create mode 100644 README.MD
create mode 100644 changecoin.png
create mode 100644 image/changeclimbspeed.png
create mode 100644 image/changejumpsspeed.png
create mode 100644 image/coin.png
create mode 100644 image/movespeed.png

C:\Data\Lab\HK 2 2025 - 2026\Game App\tilevania-3>git push
Enumerating objects: 15, done.
Counting objects: 100% (15/15), done.
Delta compression using up to 24 threads
Compressing objects: 100% (11/11), done.
Writing objects: 100% (11/11), 1.92 MiB | 937.00 KiB/s, done.
Total 11 (delta 4), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (4/4), completed with 4 local objects.
To https://github.com/BoyTay/TileVania.git
b644eb7..1e13def  main -> main

C:\Data\Lab\HK 2 2025 - 2026\Game App\tilevania-3>
```